1 - libft

kaynaklar

- [ ] https://stackoverflow.com/questions/5233514/where-to-find-stdio-h-functions-implementations
  kütüphane fonksiyonlarının kaynak kodlarını bulabileceğiniz bir so linki
- [ ] http://www.sis.pitt.edu/mbsclass/tutorial/advanced/makefile/build.htm
  makefile yazmayi öğretiyor
- [ ] https://www.geeksforgeeks.org/data-structures/linked-list/
  c'de listeler sadece buradan bile öğrenilebilir
- [ ] https://en.cppreference.com/w/c/string/byte
  c cpp öğrenmek için güzel site
- [ ] şadi evren c listesi
  büyük adam

`STUDY LIST IN LIBFT`

- [ ] c'de linked lists
  - [ ] baştan eleman ekleme
  - [ ] ortadan ekleme
  - [ ] sondan eleman ekleme
  - [ ] listenin boyutunu alma
- [ ] c'de file descriptor olayı, file handling
- [ ] c'de standart kütüphane fonksiyonları (glibc)
- [ ] c'de kütüphane yaratma
- [ ] struct ve typedef yapısı
- [ ] void pointer ve genel olarak pointerlar, pointer aritmetiği
- [ ] fonksiyon pointerları nedir
- [ ] size_t, unsigned nedir
- [ ] unsigned char nedir nasıl olur
- [ ] makefile nedir ne işe yarar `all $(NAME) clean .phony re` vs.
- [ ] bir değişkeni cast etme —> (char *)s;
- [ ] fonksiyonlar `man` ve `man 2` page'leri
- [ ] glibc fonksiyonlarına göz gezdir

- [ ] leak vs segfaultlar nasıl kontrol ediliyor, valgrind kullanımı

libft fonksiyonları ne işe yarıyor

```
ft_atoi : ascii türündeki str'yi integer'a çevirir
int ft_atoi(const char *str)

ft_bzero : s'ye baştan n kadar null koyar
oid ft_bzero(oid *s, size_t n)

ft_calloc : count adet size boyutunda nesne için yer ayırır ve ayırdıgı yeri nulla dolduru
oid *ft_calloc(size_t count, size_t size)

ft_isalnum : parametresi harf ya da rakamsa 1, değilse 0 döndürür
int ft_isalnum(int c)

ft_isalpha : parametresi harf ise 1, değilse 0 döndürür
int ft_isalpha(int c)

ft_isascii . parametresi ascii ise 1, değilse 0 döndürür
int ft_isascii(int c)

ft_isdigit : parametresi rakamsa 1, değilse 0 döndürür
int ft_isdigit(int c)

ft_isprint : parametresi printable (0-31 aralığında) ise 1, değilse 0 döndürür
int ft_isprint(int c)

ft_itoa : integer'ı char stringe çevirir
char *ft_itoa(int n)

ft_memchr : s'de c'yi buldugu zaman c dahil s'nin kalanını döndürür
void *ft_memchr(const void *s, int c, size_t n)

ft_memcmp : s1 ve s2'nin ilk n karakterini karşılaştırır, fark çıkarsa ascii farkını döndürür
int ft_memcmp(const void *s1, const void *s2, size_t n)

ft_memcpy : src'den dst'e n kadar kopyalar. aynı hafıza alanındaysa it may though overlap
void *ft_memcpy(void *dst, const void *src, int n)

ft_memmove : src'den dst'e len kadar kopyalar, aynı hafıza alanındaysa sondan kopyalar, sorun olma
oid *ft_memmove(void *dst, const void *src, size_t len)

ft_memset : b'nin ilk len karakterine c yazar
void *ft_memset(void *b, int c, size_t len)

ft_putchar_fd : dosyaya karakter yazdırır
void ft_putchar_fd(char c, int fd)

ft_putendl_fd : dosyaya string yazdırır, sonuna \n koyar
void ft_putendl_fd(char const *s, int fd)

ft_putnbr_fd : dosyaya sayı yazdırır
void ft_putnbr_fd(int n, int fd)

ft_putstr_fd : dosyaya string yazdırır
void ft_putstr_fd(char *s, int fd)

ft_tolower : büyükse küçük yapar
int ft_tolower(int c)
```

```
ft_toupper : küçükse büyük yapar
int ft_toupper(int c)

ft_split : s'yi c'lerden ayırıp 2D bir arraye atar, onu döndürür
char ft_split(char const *s, char c)

ft_strchr : s'de c'yi bulursa c dahil kalanını döndürür
char *ft_strchr(const char *s, int c)

ft_strrchr : s'de sondan geriye c'yi arar bulursa c dahil kalanı döndürür
char *ft_strrchr(const char *s, int c)

ft_strdup : s1'e önce yer ayırıp onu bir temp'e kopyalar, sonra tempi sonra döndürür
char *ft_strdup(const char *s1)

ft_striteri : s'nin tüm elemanlarını (f) fonksiyonuna tâbi tutar ama bunu string üzerinde yapar
void ft_striteri(char *s, void (*f)(unsigned int, char *))

ft_strmapi : s'nin tüm elemanlarını (f) fonksiyonuna tâbi tutar ama bunu buffer üzerinden yapar
char *ft_strmapi(char const *s, char (*f)(unsigned int, char))

ft_strjoin : s1 ve s2 stringlerini uç uca ekler
char *ft_strjoin(char const *s1, char const *s2)

ft_strlcat : src'yi dest'in sonuna atar ama destSize-1 kadar yer ayırır, null'a kesinlikle 1 byte bırakı
size_t ft_strlcat(char *dest, const char *src, size_t destSize)

ft_strlcpy : src'yi dest'e kopyalar ama maksimum destSize kadar yer ayırır
size_t ft_strlcpy(char *dest, const char *src, size_t destSize)

ft_strlen : s'nin uzunluğunu hesaplar
size_t ft_strlen(const char *s)

ft_strncmp : s1 ve s2'nin ilk n karakterini karşılaştırır, fark bulursa farkı döndürür
int ft_strncmp(const char *s1, const char *s2, size_t n)

ft_strnstr : haystack içinde ilk len karakterde needle arar,
bulursa needle dahil kalanı döndürür. strchr'den farkı char değil string arar
char *ft_strnstr(const char *haystack, const char *needle, size_t len)

ft_strtrim : s1'in başından ve sonundan set ve türevlerine denk gelirse siler
char *ft_strtrim(char const *s1, const char *set)

ft_substr : str'nin start. indisinden itibaren len karakter kadar yazdırır
char *ft_substr(const char *str, unsigned int start, size_t len)

ft_lstnew : listeye yeni eleman ekler
t_list *ft_lstnew(oid *content)

ft_lstsize : listenin eleman sayısını verir
int ft_lstsize(t_list *lst)

ft_lstlast : listenin son elemanını verir
t_list *ft_lstlast(t_list *lst)

ft_lstadd_front : listeye baştan eleman ekler
void ft_lstadd_front(t_listalst, t_list *new)

ft_lstadd_back : listeye sondan eleman ekler
void ft_lstadd_back(t_listalst, t_list *new)

ft_lstdelone : listeden bir eleman siler yerini de free() ile serbest bırakır
void ft_lstdelone(t_list *lst, void (*del)(void *))
```

```
ft_lstclear : listeden bir dizi eleman siler
void ft_lstclear(t_listlst, void (*del)(void *))

ft_lstiter : listenin tüm elemanlarını (f) fonksiyonuna tâbi tutar ve döndürür
void ft_lstiter(t_list *lst, void (*f)(void *))

listenin tüm elemanlarını önce buffera atar, onu (f) fonksiyonuna tutar ve döndürür
t_list *ft_lstmap(t_list *lst, void *(*f)(void *), void (*del)(void *))
```

- c dilinde kendi kütüphanemizi yazmak için yapmamız gerekenler

  ▼ bir klasörde .h uzantılı bir dosya oluşturuyoruz. içeriği

```
#ifndef MY_LIBRARY_H
#define MY_LIBRARY_H

# include <libraries_you_want_to_include.h>

/* prototypes of functions you'll use within this library

int ft_strlen(char *str);
void ft_error(int errNo);
char *strrev(char *str);

*/

#endif
```

  ▼ daha sonra .c uzantılı dosyalarında #include “my_library.h” diyerek kullanabilirsin

  ▼ .c’lerde başında static koymadığın tüm fonksiyonların prototiplerini .h’a dahil etmelisin.

  ▼ bu kütüphane mantıgını kullanabilmek için my_library.h kullandıgın tüm dosyaları `cc main.c deneme1.c deneme2.c` diyerek beraber derlemelisin (ya da makefile kullanmalı)

POINTERS

- bilgisayarda bir değişken tanımladığın zaman bu değişkenin bilgisayarın beyninde 2 değeri vardır: [1] sağ değer [2] sol değer (L/R-value). sol değer bu değişkenin tutulduğu adres, sağ değer ise değişkenin türü neyse o türden değeridir. mesela

```
int a = 30;
```

diyelim. 32bit bir bilgisayarda tek hücrede tam saklanabilir bu integer değişkeni (çünkü int’in boyutu 4 byte yani 32bit). bu durumda a değişkeninin sol-değeri (L-value) 0x20400 (farazi), sağ-değeri (R-value) ise 30 olacaktır.

- kod yazarken kullandığımız değişkenler bilgisayarda RAM denen geçici hafızada tutulur. bu RAM milyonlarca küçük hücreden oluşuyor gibi düşünülebilir. bu hücrelerin her biri 8, 32, 64… bit olabilir. yani her bir hücre maksimum 32 ya da 64 bit veri tutuyor. atadığımız verileri unutmamak için

bilgisayar bilgileri bu hücrelere kaydeder. mesela standart mimaride bir `int` 4 byte yani 32 bittir. kod yazarken bir `int a = 13;` tanımladıgımız vakit bilgisayar 32 tane art arda küçük hücreyi bu bilgiyi saklamaya ayırır. daha sonra bu a değişkenine bir pointer atadığımız zaman bu pointer değişkeni ram'deki bu 32 bitlik blogun ilk bitinin hexadecimal formatta sayısal karşılığını gösterir hale gelir. yani pointerlar tanımladığımız değişkenlerin, fonksiyonların, structların vs. ram'de saklandığı hücrelerin adresleridir.

- pointer adresleri hexadecimal türündedir ve pointer tanımlamalarında değişkenlerin pointer olduklarını göstermek için başlarına * koyarız. * operatörünün iki görevi vardır: [1] pointer tanımlarken kullanırız [2] tanımladığımız pointerın başında kullanılırsa pointerın tuttuğu değeri alırız. buna dereferencing denir.

```
int a = 30;
int *aPtr = &a; // birinci görev
*aPtr == 30;    // ikinci görev
```

- bir değişkenin adresini atamak istediğimiz zaman da & operatörünü kullanacağız. & operatörü başına geldiği şeyin hafızadaki adresine ulaşmak için kullanılır. mesela 0x10a2500 bir adres olabilir. pointer adresleri printf( ) ile yazdırılırken %p ile yazdırılır:

```
int main()
{
    int a = 14;

    // & ile adres yazdırmanın birinci yolu
    int *aPtr = &a;
    printf("%p\n", aPtr);  >>> 0x7ffee7d4c72c

    // ikinci yolu
    printf("%p\n", &a);    >>> 0x7ffee7d4c72c

    // adresin adresini aliyoruz, yanlış
    printf("%p\n", &aPtr); >>> "pointerın tutuldugu adres gelir, a'nın değil"

    // dereferencing
    int a_value = *aPtr;
    printf("%d\n", a_value); >>> 14
}
```

- bit comes from binary digits.
- & operatörü * operatörünün karşıtıdır. birbirlerini götürürler

```
int main()
{
    int a = 23;
    int *aPtr = &a;
    printf("%d\n", *&*&*aPtr);
}
```

23. &*’ler birbirini götürür * kalır o da dereference ettiği için değerini alır.

- örnek:

```
*ptr = 65;
```

demek ptr isimli pointera adres olarak 65 hexini vermek demektir.

```
int a = *ptr;
```

ise a’ya ptr pointerının değerini (tuttuğu adresteki değeri) atar.

- örnek:

```
int main()
{
    int *a;
    a = malloc(sizeof(int) * 10);
    if (!a)
        return 0;
    a[0] = 10;
    a[1] = 20;
    a[2] = 30;
    a[3] = 40;            // şu an a[] = {10, 20, 30, 40};
    int b = *a + 3;       // *a demek a'nın ilk elemanı demektir o da 10
    printf("b: %d\n", b); // 13 yazdırır
}
```

*(a+3) deseydik 40 yazdırır çünkü a’nın 3 ötelenmiş elemanının değerini atardı b’ye

- bir pointerın gösterdiği adresteki değeri 1 arttırmak istiyosan

```
*ptr += 1;
*ptr = *ptr + 1;
(*ptr)++;

int main()
{
    int a = 10;
    int *aPtr = &a;
    (*aPtr)++;
    (*aPtr)++;
    printf("%d\n", a); >>> 12
}
```

diyebilirsin ama (*ptr)++ ile *ptr++ çok farklı şeyler oldugunu unutmamalısın. ilki değeri 1 arttırırken diğeri bir arttırılmış adresteki değeri aliyor çünkü ++ operatörünün önceliği *’den yüksek.

- iki pointer arası adres atayabilirsin:

```
int a=10, b=20;
int *aPtr=&a, *bPtr=&b;
aPtr = bPtr;
printf("%d\n", *aPtr); >>> 20
```

- birden fazla pointer türü vardır
  - int pointer: integer adresi tutan pointerlar

    ```
    int a = 10;
    int *aPtr = &a;
    ```

  - char pointer: char array'leri oluşturmak için kullanılır

    ```
    char *a = "bugra";
    ```

    char pointerlar arrayde her biri char türünde olan hafıza bloğunun başlangıç adreslerini tutarlar. yani c'de `string` diye bir şey yok, her elemanı charlardan oluşan char dizileri - `s[], *s` var.

  - void pointer (aka generic pointer): belli bir veri tipi olmayan pointer, her şeyle kullanılabilir.

    ```
    void *p = NULL;
    ```

  - null pointer: null atanmış pointer

    ```
    int *ptr = NULL;
    ```

  - wild pointer: ilklendirilmemiş pointerlardır. haliyle garbage value tutarlar. kullanımı risklidir

    ```
    int *myPtr;
    ```

  - dangling pointer
  - complex pointer
  - double-pointer
  - near pointer
  - far pointer
  - huge pointer
- bir char dizisi tanımladıktan sonra (`char *arr`) artık o arrayin adı (`arr`) pointer'ının sembolü yerine geçer. örnek:

```
int main(void)
{
  char *a = "bugra";
  printf("%c\n", *(a));     >>> b
  printf("%c\n", *(a + 3)); >>> r
  printf("%c\n", *a + 2);   >>> d
}
```

- bu blogu pointer aritmetiğiyle modifiye edebilirsin:

```
int main()
{
    char *str = "bugra";
    str++;
    str++;
    printf("%s", str);
} >>> gra
```

- pointerların içinde değer yani value tutmazlar adres tutarlar. bu adresin işaret ettiği şey value tutabilir ya da tutmayabilir. mesela printf("%d", *p") ifadesi p'nin adresine git o adresteki değerini yazdır demektir. p'nin adresini (0x032df132…) değil
- `[]` bir pointer operatörüdür. arrayler'de [ ]'ler adresteki değere gitmek için kullanılır ve array indeksleri 0'la başlar.

```
array[] = {3, 5, -10, 20, 150};
a[3] >>> 20;
```

[ ] içine yazdığın sayı kadar diziye öteleme işlemi uygular. mesela arr[3] demek *(arr + 3) demektir.

bir array çok boyutlu (yani matris) de olabilir:

```
array[][] = {{1, 2, 3}, {10, 20, 30}, {40, 50, 60}};
array[2][0] >>> 40
```

- örnek:

```
int *p1, p2;
```

demek p1 isimli bir pointer ve p2 isimli normal bir int değişkeni tanımlar

- `call by reference` vs. `call by value`:

```
void addOne_not_works(int n)
{
    n++;
```

```
}

void  addOne_works(int *n)
{
  (*n)++;
}

int main(void)
{
  int n;

  n = 5;
  addOne_not_works(n); >>> 5
  addOne_not_works(n); >>> 5
  addOne_works(&n);    >>> 6
  addOne_works(&n);    >>> 7
  addOne_works(&n);    >>> 8
  printf("%d\n", n);
}
```

bunun sebebi fonksiyona çalışmayan fonksiyonda parametre olarak değişkenin değerini gönderiyoruz haliyle fonksiyon bitince değeri de sıfırlanıyor çünkü adresindeki değerine bir şey yapmıyoruz. *n ve &n ile kullandığımzda ise adresteki değeri arttirdiği için fonksiyon scopu bitse de değişken arttırılmış oluyor.

- gcc ile derledikten sonra default olarak a.out dosyası oluşur (windows'ta a.exe). kendin isim vermek istiyorsan → gcc deneme.c -o run dersen run ismini verir a.out'a. bunu calistirmak icin de ./run demelisin.

CASTING

- iki tip cast etme vardır
  - implicit: when data type is converted automatically by the C program:

    ```
    int a = 98;
    printf("%c", a); >>> b
    ```

    %c dediğimiz için otomatik olarak derleyici veri tipini char'a uyarladı ve char karşılığın verdi
  - explicit (aka typecasting): kendimiz değiştiriyoruz:

    ```
    int a = 98;
    printf("%c", (char)a);
    ```

- bir fonksiyonda void olarak gelen parametreyi kullanabilmek için typecast etmelisin:

```
int myFun(void *src, char *dst)
{
  src = (char *)src;
  ...
}
```

STRUCTURES

- C structures are special, large variables which contain several named variables inside. The most basic example of structures are points, which are a single entity that contains two variables - x and y. Let's define a structure:

```
struct    myStruct
{
  int   age;
  char  *name;
  float salary;
};

int main(void)
{
  struct myStruct ben;

  ben.age = 10;
  ben.name = "bugra";
  ben.salary = 128.500;
  printf("%d\n", ben.age);      >>> 10
  printf("%s\n", ben.name);     >>> bugra
  printf("%.3f\n", ben.salary); >>> 128.500
}
```

Typedefs allow us to define types with a different name - which can come in handy when dealing with structs and pointers. In this case, we'd want to get rid of the long definition of a point structure. typedef'li kullanırsan her seferinde `struct myStruct ben;` yerine `ms ben;` demen yeterli olur:

```
typedef struct myStruct
{
  int   age;
  char  *name;
  float salary;
} ms;

int main() {
  ms ben;
  ben.age = ---;
  ben.name = ---;
  ben.salary = ---;
}
```

- hem `->` hem `.` operatörü structın değerlerini atamak ya da almak için kullanılır ama aralarındaki fark şudur: eğer struct yapısını pointer şeklinde kurduysan -> kullanmalısın, normal kurduysan . kullanmalısın:

```
// -> kullanımı
struct Node
{
    int age;
```

```
    char *name;
};

int main()
{
    struct Node *ben = malloc(sizeof(struct Node));
    ben->age = 21;
    ben->name = "bugrahan";
    printf("%d\n", ben->age);
    printf("%s\n", ben->name);
}

// . kullanımı
struct Node {
    int age;
    char *name;
};

int main() {
    struct Node ben;
    ben.age = 21;
    ben.name = "bugrahan";
    printf("%d\n", ben.age);
    printf("%s\n", ben.name);
}
```

pointerlı tanımladığın yerde . kullanmak, pointersız tanımladığın yerde de → kullanmak hata sebebidir.

DYNAMIC MEMORY ALLOCATION

- Dynamic allocation of memory is a very important subject in C. It allows building complex data structures such as linked lists. Allocating memory dynamically helps us to store data without initially knowing the size of the data in the time we wrote the program. To allocate a chunk of memory dynamically, we need to have a pointer ready to store the location of the newly allocated memory. We can access memory that was allocated to us using that same pointer, and we can use that pointer to free the memory again, once we have finished using it. örneğin şöyle bi yapımız olsun:

```
typedef struct
{
    char *name;
    int age;
} person;

/* to allocate a new person in the myperson argument,
    we use the following syntax : */

int main()
{
    person *myperson = (person *)malloc(sizeof(person));
    myperson->name = "bugra";
    myperson->age = 21;
    printf("%s is %d years old.\n", myperson->name, myperson->age);
    free(myperson);
}
```

This tells the compiler that we want to dynamically allocate just enough memory to hold a person struct in memory and then return a pointer of type person to the newly allocated data. The memory allocation function malloc() reserves the specified memory space. In this case, this is the size of person in bytes. The reason we write (person *) before the call to malloc() is that malloc() returns a "void pointer," which is a pointer that doesn't have a type.Writing (person *) in front of it is called typecasting, and changes the type of the pointer returned from malloc() to be person. we can assess its elements with -> operator.

- örnek2:

```
str2 = (char *)malloc(sizeof(char) * str.len);
if (!str2)
  return 0;
```

yani malloc() fonksiyonu `yeri_ayrilacak_degisken = (return_türü)malloc(ne_kadar_yer);` olarak kullanılıyor.

- c yazarken herr malloc() kullanımı sonrasında 2 şeyi kontrol etmelisin
  - yer açılamamışsa return etmelisin

    ```
    str = malloc(----);
    if (!str)
      return 0;
    ```

  - açtığın şeyi free()'lemelisin

    ```
    {
      str = malloc(----);
      ----
      ----
      ----
      free(str);
      return 0;
    }
    ```

- malloc'u ve malloc kontrolünü tek satırda halledebilirsin:

```
if (!(str = malloc(sizeof(char) * 10)));
  return 0;
```

MAKEFILE

- libft'de çok sayıda .c kullandığımız için make ile derliyoruz. make işletim sistemlerinde kaynak kodları tek seferde ve çoklu derlemeye yarayan bir metot. bu metodun unix/mac'te en çok kullanılan örneği ise Makefile. libft'de kullandığımız makefile:

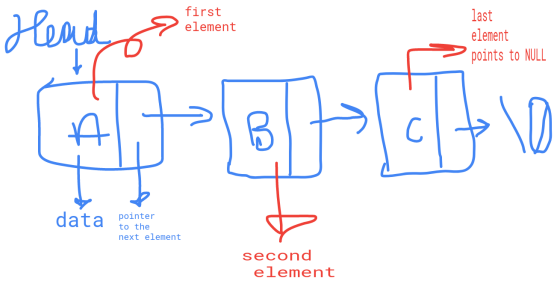
```
# ************************************************************************** #
#                                                                            #
#                                                        :::      ::::::::   #
#    Makefile                                          :+:      :+:    :+:   #
#                                                    +:+ +:+         +:+     #
#    By: bkaramol <bkaramol@42istanbul.42.fr>          +#+  +:+       +#+     #
#                                                +#+#+#+#+#+   +#+           #
#    Created: 2022/07/05 04:01:36 by bkaramol          #+#    #+#             #
#    Updated: 2022/07/16 19:59:39 by bkaramol         ###   ########.fr       #
#                                                                            #
# ************************************************************************** #

#   makefile tekniği çok sayıda kodu kolayca derleyip debug etmeye yarar.
#   yani tek tek gcc *.c yazmamıza gerek kalmıyor. gcc'nin çalışma mantığı
#     1) preprocessing (standart çıktı) [gcc -E]
#       * yorumları siler
#       * makroları açar
#       * .h uzantılı kütüphaneleri import eder
#         bunun için direkt olarak kütüphaneyi kopyalar
#         koda yapıştırıp çalıştırır
#     2) compiling (derleme) [gcc -S]
#       * assembly koduna çevirir
#       * böylelikle asd.s (assembly uzantısı) oluşur
#     3) assembly (bir araya getirme) [gcc -c]
#       * assembly kodu makina diline (0-1) çevrilir
#       * böylece .o uzantılı object dosyaları oluşur
#         bu dosyalar makine diline en yakın aşamadır
#     4) linking (bağlama)
#       * nesne kodları çalıştırılabilir bir dosyaya linklenir
#       * bu işlemin sonucunda nihai çalıştırılabilir dosya
#         elde edilir (.exe ya da .out veya uzantısız)

# extension aramaya vsc'de ext:s,c,o vs. yazarsan o uzantıyla ilgili
# paketler gelir
# değişkenleri yazarken $() içine yazıyoruz
# make ... demek ... komutunu çalıştır demek
# direkt make yazarsan ilk komutu çalıştırır

# burada kaynak kodlarımızı listeliyoruz
SRCS      = ft_atoi.c ft_bzero.c ft_calloc.c ft_isalnum.c ft_isalpha.c ft_isascii.c \
          ft_isdigit.c ft_isprint.c ft_itoa.c ft_memchr.c ft_memcmp.c ft_memcpy.c \
          ft_memmove.c ft_memset.c ft_putchar_fd.c ft_putendl_fd.c ft_putnbr_fd.c \
          ft_putstr_fd.c ft_split.c ft_strchr.c ft_strdup.c ft_striteri.c ft_strjoin.c \
          ft_strlcat.c ft_strlcpy.c ft_strlen.c ft_strmapi.c ft_strncmp.c ft_strnstr.c \
          ft_strrchr.c ft_strtrim.c ft_substr.c ft_tolower.c ft_toupper.c

# bu komut ile .c uzantılıları .o uzantıya çeviriyoruz
# .o(bject) dosyası ne demek? makine dili ile C dili arasındaki levelde bir dosya
# türüdür. derleyici, ingilizce kelimelerle yazılan kodu 0-1 diline çevrilmeden önce
# bu dosyalara dönüştürür
OBJS      = $(SRCS:.c=.o)

# bonusları ekliyoruz
BONUS     = ft_lstadd_back.c ft_lstadd_front.c ft_lstclear.c \
          ft_lstdelone.c ft_lstiter.c ft_lstlast.c \
          ft_lstmap.c ft_lstnew.c ft_lstsize.c

BONUS_OBJS    = $(BONUS:.c=.o)

# aliaslar ekliyoruz
CC        = gcc
```

```
# make fclean dediğimizde rm -f komutunu kullanacak
RM        = rm -f

# otomatik olarak bu flaglerle derlemesini istiyoruz
CFLAGS      = -Wall -Wextra -Werror -I.

NAME      = libft.a

# iki noktadan önce yazdıkların kuraldır
# yanına yazdıkların kuralın geçerli olacağı şeydir
# altına indented yazdıkların da o kuralı gerçekleştirecek komutlardır
# kural: nesne
#  komutlar
# komutlar sonucu kural oluşur, kuralın oluşabilmesi için nesneye ihtiyaç var
# makefile soldan sağa ve yukarıdan aşağıya çalışır. sıralamalar önemlidir

# all -> tüm kuralları çalıştıran kök kural
# burada neden direkt ar rcs komutunu all'ın altında yazmadık?
# çünkü biz en sonda $name isimli bir dosya oluşsun istiyoruz ki relink olmasın
# yani her make dendiginde makefile tekrar tekrar çalışmasın. bunun için ana kuralın
# çıktısı isminde bir dosya olmalı, bunu da all'ı dolaylı yazarak sağlıyoruz
all:      $(NAME)

# başlarındaki $ onların değişken olduğunu söylüyor
# ar means archive. yoksa oluşturuyor.
# r --> replace the content in the archive
# c --> create a new archive
# s --> Write an object-file index into the archive
$(NAME): $(OBJS)
        ar rcs $(NAME) $(OBJS)

# make clean: derleme sonrası oluşan ara dosyaları siler
clean:
        $(RM) $(OBJS) $(BONUS_OBJS)

# make fclean: derleme sonrası oluşan tüm dosyaları siler (fullclean)
fclean: clean
        $(RM) $(NAME)

# make re: kütüphaneyi yeniden çalıştırır. önce fclean ile
# her şeyi siler sonra all ile yeniden yapar
re:     fclean all

bonus:  $(OBJS) $(BONUS_OBJS)
        ar rcs $(NAME) $(OBJS) $(BONUS_OBJS)

# C clean kuralını çalıştırmak istesin. aynı adda bir başka klasör ya da dosya varsa
# ona gitmesin kural olan clean'i anlasın diye bunu yazıyoruz. mesela clean diye bi dosyan
# olsun. make clean dediğinde .o noktalarını silme işlemi değil clean'ı derleme işlemi yapacaktır
# eğer .phony'ye clean ismini dahil etmezsen. .gitignore gibi düşünebilirsin
.PHONY:     all clean fclean re bonus
```

derledikten sonra her .c'nin .o'su + olarak da libft.a çıkacaktır. birden fazla dosyanın kullanıldığı bir dosyayı kısaca derlemek için `dosya.c libft.a` diyebilirsin. burada derleme işlemini yaparken cc ya da gcc değil ar rcs yani arşiv oluşturma metodunu kullanıyoruz çünkü libt'de main( ) fonksiyonu yok o yüzden derleyip executable program oluşturmaya çalışmak hata verir.

LINKED LIST

- linked list yani bağlı listeler bir veri yapısı türüdür. içerlerinde farklı türden verileri tutmaya yararlar. peki listeler nasıl çalışıyor? listeler arrayler gibi hafızada sıralı hücrelerde değil dağınık hücrelerde tutuluyor. her hücre hem bir veriyi hem sonraki hücrenin bilgisini saklıyor.
  - each element of a list contains
    - A Data Item (we can store integer, strings, or any type of data).
    - Pointer (Or Reference) to the next node (connects one node to another) or An address of another node



  listelerin avantajı kolaylıkla veri eklenip çıkartılması. hatta ileri düzey listelerde birden fazla listeyi birbirine bağlayabiliyorlar. dezavanyaji ise access time'ı biraz uzun çünkü location (indis) kesin değil arraylerin aksine. o yüzden arraylerde spesifik bir elemana ulaşmak hem daha kolay hem daha kısa sürüyor.
- liste türleri
  - singly linked list: her eleman bir sonrakini gösteriyor son eleman null'u gösteriyor. libft bonusunda bu tip listeleri kullanıyoruz. işlem olaraksa baştan ekleme sondan çıkarma vs. gibi klasik liste problemleri çözmemiz isteniyor.
  - doubly linked list: her eleman bir sonrakini ve bir öncekini gösteriyor son eleman null'u ve bir öncekini gösteriyor.
  - circular linked list: her eleman bir sonrakini gösteriyor son eleman ilk elemanı gösteriyor.
- A linked list is represented by a pointer to the first node of the linked list. The first node is called the head of the linked list. If the linked list is empty, then the value of the head points to NULL.
- In C, we can represent a node using structures. Below is an example of a linked list node with integer data in c:

```
// A linked list node
struct Node {
    int data;
    struct Node* next;
};
```

- `memchr` vs. `strchr` farkı: strchr stops when it hits a null character but memchr does not; this is why the former does not need a length parameter but the latter does. Functionally there is no difference in that they both scan an array / pointer for a provided value. The memchr version just takes an extra parameter because it needs to know the length of the provided pointer. The strchr version can avoid this because it can use strlen to calculate the length of the string. Differences can popup if

you attempt to use a char* which stores binary data with strchr as it potentially won't see the full length of the string. This is true of pretty much any char* with binary data and a str* function. For non-binary data though they are virtually the same function. `strchr` expects that the first parameter is null-terminated, and hence doesn't require a length parameter. `memchr` works similarly but doesn't expect that the memory block is null-terminated, so you may be searching for a \0 character successfully.

- `memmove` vs. `memcpy` farkı: With memmove, the destination cannot overlap the source at all. With memcpy it can.

E.g. if `memcpy` copies "front to back" and the memory blocks are aligned as this

```
[---- src ----]
            [---- dst ---]
```

copying the first byte of `src` to `dst` already destroys the content of the last bytes of `src` before these have been copied. Only copying "back to front" will lead to correct results.

Now swap `src` and `dst`:

```
[---- dst ----]
            [---- src ---]
```

In that case it's only safe to copy "front to back" as copying "back to front" would destroy `src` near its front already when copying the first byte.

Unless you know for sure that `src` and `dst` don't overlap, call `memmove` as it will always lead to correct results and is usually as fast as that is possible for the copy case you require.

If you know for sure that `src` and `dst` don't overlap, call `memcpy` as it won't matter which one you call for the result, both will work correctly in that case, but `memmove` will never be faster than `memcpy` and if you are unlucky, it may even be slower, so you can only win calling `memcpy`.

```
int main(void)
{
    char s1[] = "123456789abcd";
    char *s2, *s3;
    s2 = s1;
    s3 = s1;

    printf("initially s1:  %s\n", s1); >>> "123456789abcd"
    memcpy(s2 + 5, s2, 7);
    printf("memcopied: s2: %s\n", s2); >>> "123451234567d"
    memmove(s3 + 5, s3, 7);
    printf("memmoved s3:   %s\n", s3); >>> "123451234512d"
    return 0;
}
```

FILE DESCRIPTORS

- In C, you can perform four major operations on files, either text or binary:
  1. Creating a new file

2. Opening an existing file
3. Closing a file
4. Reading from and writing information to a file

- A file descriptor is a positive number that uniquely identifies an open file in a computer's operating system. The descriptor is identified by a unique non-negative integer, such as 0, 12, or 567. At least one file descriptor exists for every open file on the system. File descriptors were first used in Unix, and are used by modern operating systems including Linux, macOS, and BSD. In Microsoft Windows, file descriptors are known as file handles. ilk 3 FD default olarak şunlardır:
  - 0 - standard input (stdin): The default data stream for input, for example in a command pipeline. In the terminal, this defaults to keyboard input from the user.
  - 1 - standard output (stdout): The default data stream for output, for example when a command prints text. In the terminal, this defaults to the user's screen.
  - 2 - standard error: The default data stream for output that relates to an error occurring. In the terminal, this defaults to the user's screen.
- `fcntl.h` kütüphanesini kullanırlar.
- bir dosya açmak istediğinde çekirdek önce
  - o dosyaya erişim izni kontrol eder. varsa
  - global file table'da bir entry yaratır
  - programa açması için dosyanın konumunu gönderir
- bazı I/O fonksiyonları:
  - `int open(char *path, int flags)` open fonksiyonu eğer varsa path isimli dosyayı açar, yoksa flagler yardımıyla oluşturur. return olarak açtığı dosyanın fd'sini döndürür dolayısıyla bir dosyanın fd'sini integer'a atamana yarar.

```
int main() {
  int my_fd = open("abc.txt", O_RDWR);
}
```

  - `ssize_t read(int fd, char* buf, size_t sz)` Read data upto count bytes from a file whose file descriptor is fd and store the results in buf pointer. It returns the number of bytes read. In case of any error, it returns -1.

```
int main()
{
    int fd = open("asd.txt", O_RDWR);
    char *c = (char *)malloc(1000);
    int chk = read(fd, c, 1000);
    printf("%s\n", c);
}  >>> content of asd.txt
```

open() ile asd.txt dosyasının fd'sini öğrendik. bunu daha sonra read() fonksiyonunda kullanarak asd.txt'ten c stringine okuma yapmada kullandık.

- `ssize_t write(int fd, const char* buf, size_t sz)` write fonksiyonu buf char stringinden fd id'li dosyaya sz byte kadar okuma yapar. Normally, write returns sz, but as with read, it might return less. bunlara short read/write denir.

```
void  ft_putchar_fd(char c,  int fd)
{
  write(fd, &c, 1);
}

void ft_putstr_fd(char *s, int fd)
{
  write(fd, s, strlen(s));
}

int main()
{
    int x = open("bugra.txt", O_CREAT | O_RDWR);
    ft_putchar_fd('s', x);
    ft_putchar_fd('\n', x);
    ft_putstr_fd("bugra kara", x);
}
```

burada bugra.txt dosyasına s charını yazdırdık. eğer x yerine 0, 1, 2 kullansaydık terminale yazdırırdı.

- `int close(int fd)` fd id'li dosyayı kapatır.

```
int main()
{
  close(fd);
}
```

- `int creat(char *filename, mode_t mode):` filename konumunda mode izinleriyle dosya yaratır. it returns the file descriptor that points to that file in the file table entry. In case of any error, it returns -1.

```
int new_file = creat("asd.txt",0600);
```

bazı flagler:

O_RDONLY	read only
O_WRONLY	write only
O_RDWR	read and write

O_CREAT	create file if it doesn’t exist
O_EXCL	prevent creation if it already exists
__O_LARGEFILE	used for opening large files(>1GB)
O_TRUNC	Truncate the contents of the file, if the file already exists

- `file descriptor` vs. `file pointer` farkı: A file descriptor is a low-level integer "handle" used to identify an opened file (or socket, or whatever) at the kernel level, in Linux and other Unix-like systems. You pass "naked" file descriptors to actual Unix calls, such as read(), write() and so on. A FILE pointer is a C standard library-level construct, used to represent a file. The FILE wraps the file descriptor, and adds buffering and other features to make I/O easier. You pass FILE pointers to standard C functions such as fread() and fwrite(). /

---

end